DevOps, Software Evolution and Software Maintenance (BSc)

Report

ITU DevOps 2025 — Group I

Benjamin Ormstrup beor@itu.dk

Cecilie Amalie Wall Elkjær ceel@itu.dk

Johan Ingeholm jing@itu.dk

Marcus Frandsen megf@itu.dk

Philip Rosenhagen phro@itu.dk

May 17, 2026

Contents

Each section of this report, has been written and edited by all group members.

1 System's perspective

1.1 Design and architecture

When we first began the project, we opted to use our Chirp! project from the BDSA course. There were multiple reasons for this. Firstly, Helge recommended using Chirp!, and secondly, all group members were already familiar with the codebase, which we believed would make it easier to solve problems, understand each other's code, and improve maintainability.

We dockerized our application in order to deploy it across multiple servers, as illustrated in Figure 1, which presents a simplified overview of the deployment architecture and therefore does not include every container used within the system. To separate responsibilities within the system, we utilized multiple DigitalOcean droplets dedicated to services such as the web server, API server, database, monitoring infrastructure, and load balancers. By separating the system components, it became easier to deploy, update and monitor without affecting the rest of system.

As shown in Figure 2, the system also utilizes load balancing to distribute incoming traffic between multiple application servers. This improves our system's scalability and availability by ensuring that requests can be handled across different server instances instead of relying on a single deployment.

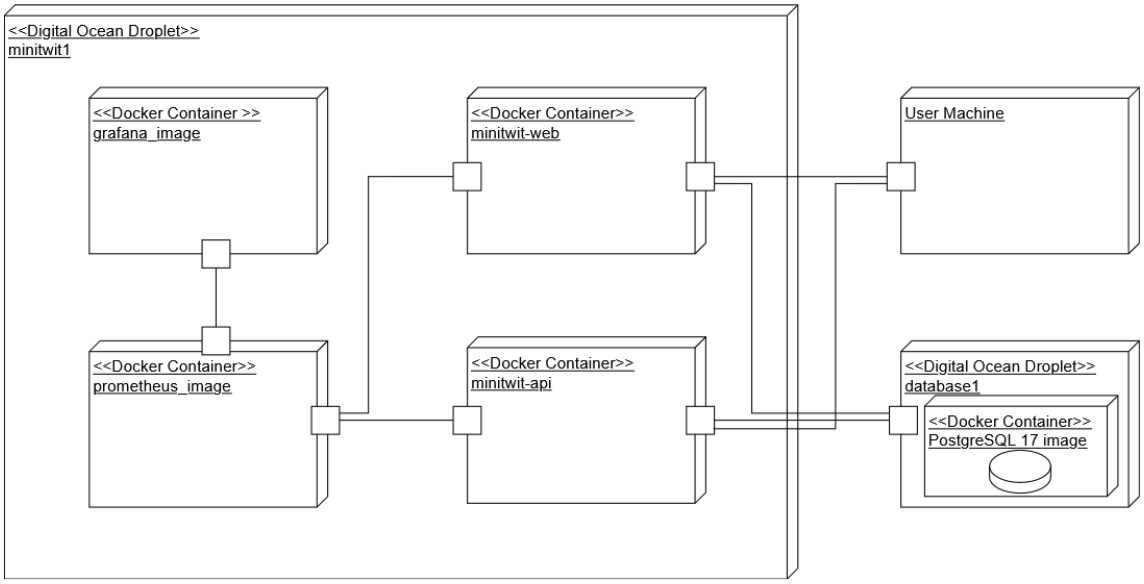

Figure 1: MiniTwit Deployment Diagram

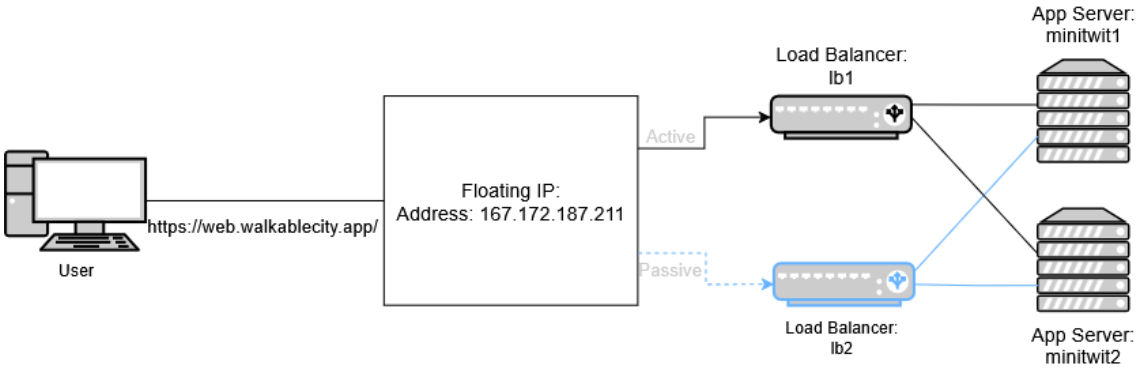

Figure 2: MiniTwit Load Balance Deployment Diagram

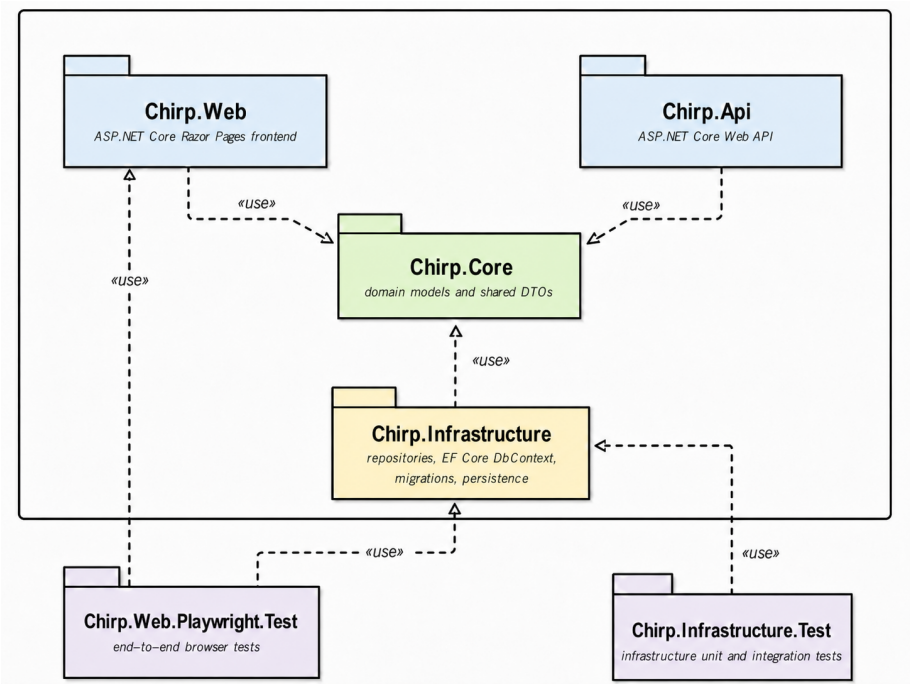

Figure 3: MiniTwit Package Diagram

1.2 Dependencies

The system depends on several technologies and tools across development and operations.

- The main implementation technologies are: C#, ASP.NET Core, Razor Pages, ASP.NET Identity, EF Core, and PostgreSQL.
- Deployment depends on: Docker, Docker Compose, Terraform, Digital Ocean, nginx, and Keepalived.
- For operations and observability, the system uses: Prometheus for metrics, Grafana for dashboards, and Loki together with Alloy for centralized logging.

- For quality assurance, the project uses: xUnit, Playwright, Test containers, Coverlet, StyleCop Analyzers, and GitHub Actions.

1.3 Flow

Below is a sequence diagram, showing the dynamic view and flow of a user posting a message. It shows the sequence of calls for a message being posted on our application through the web.

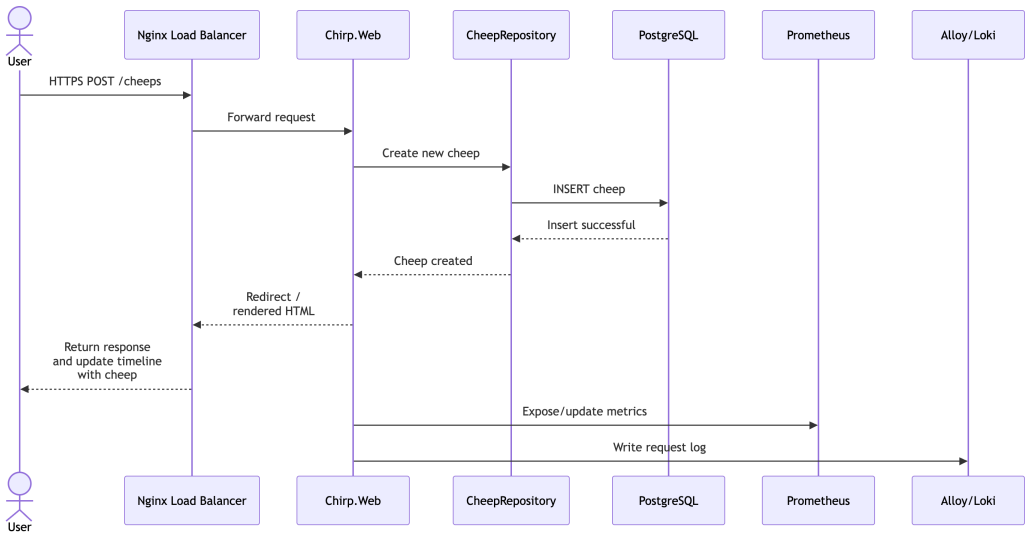

Figure 4: MiniTwit Sequence Diagram

1.4 Current state

As shown in Figures 5 and 6, the reliability, maintainability and security ratings achieved the highest possible score of A. There are however three security hotspots, earning us our lowest score of C. For a more detailed explanation of security within the system, see Section 2.4. Furthermore, the code duplication metric is low at 4.2%, indicating a limited amount of duplicated code.

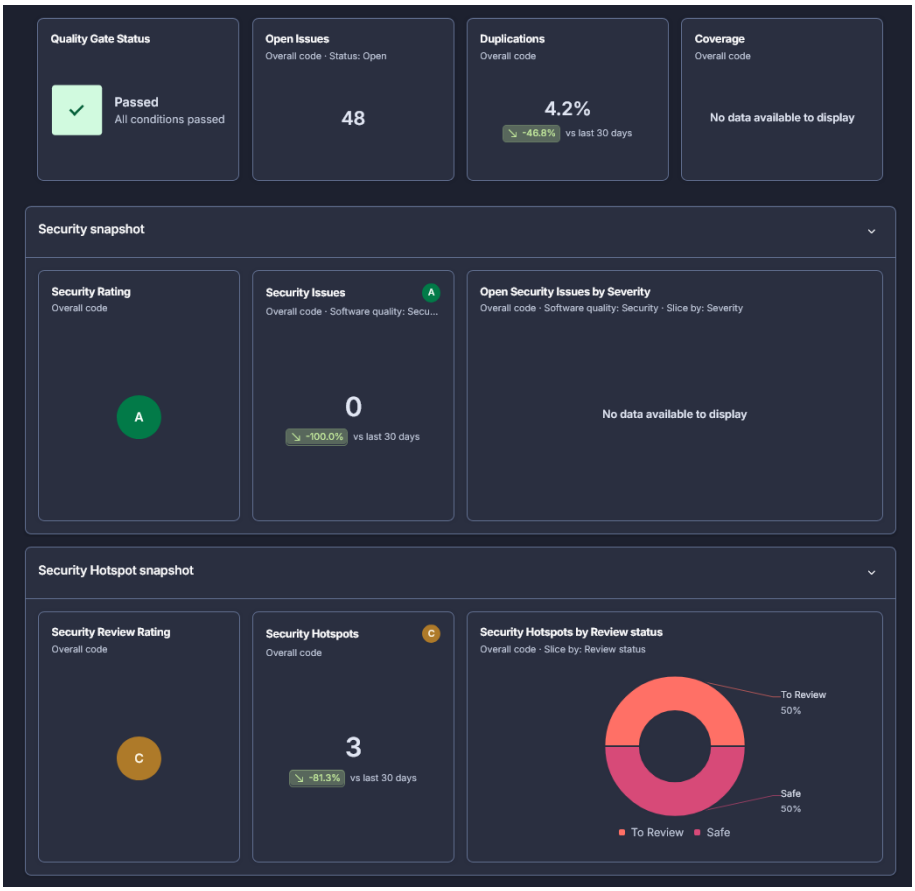

Figure 5: SonarQube Security Snapshots

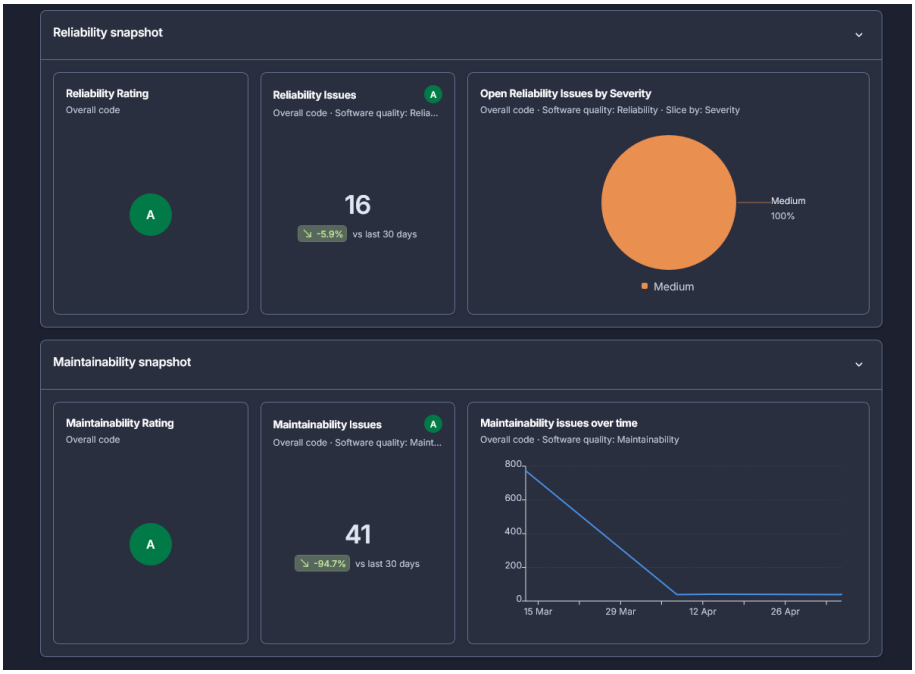

Figure 6: SonarQube Reliability and Maintainability Snapshots

2 Process' perspective

2.1 CI/CD Pipeline, Release, and Deployment

This section describes how a change goes from development to a running production system. It starts in the repository, where new features, bug fixes and other updates are added through a shared Git workflow. From there, GitHub Actions handles the CI/CD pipeline.

Whenever code is pushed to the main branch or submitted through a pull request, the pipeline restores dependencies, builds the .NET solution, and runs automated tests. At the same time, static security checks are run to catch potential weaknesses.

When a version is ready, the pipeline also handles the release. Tagged versions produce builds for different operating systems, which makes it easy to connect a specific version of the code to the artifacts that were shipped. This improves traceability and makes it clear what has been delivered.

Deployment is the final step. The application runs in containers, and the infrastructure is defined using Terraform. Terraform provisions the required resources, such as the database and application instances. After that, Docker Compose is used to start and update the services. Because the entire environment is defined in code, deployments are consistent and repeatable rather than being performed manually, which reduces the risk of human error. As part of the deployment process, Trivy scans the Docker images for high and critical vulnerabilities. If it detects any such weaknesses, the scan fails and the deployment process is stopped before the images are pushed and the services are updated.

Below is a diagram showing the flow of our release-and-deploy workflow:

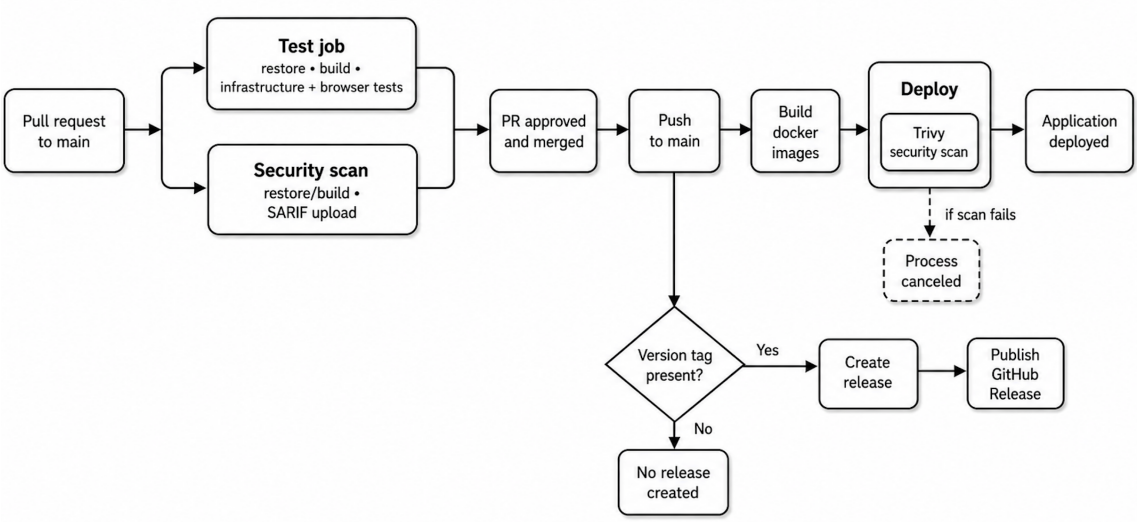

Figure 7: Release-and-deploy Workflow

2.2 Monitoring

Monitoring is based on Prometheus and Grafana. The web application and API expose metrics that Prometheus collects at regular intervals. This gives us a continuous view of how the system is behaving and helps detect issues like downtime or unusual traffic patterns. Grafana is used to display this data in dashboards, making it easier to visualize as well as understand what is going on.

Our Monitoring Dashboards:

Figure 8: MiniTwit Example Dashboard

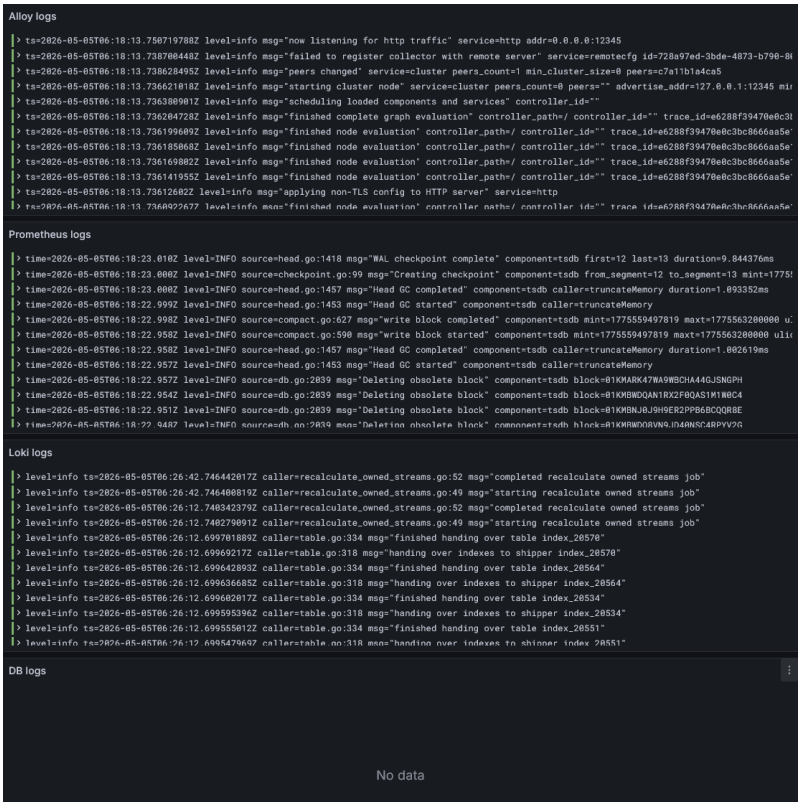

Figure 9: MiniTwit Logging Dashboard

2.3 Logging and Log Aggregation

While monitoring shows overall trends, logging gives a more detailed picture of what is actually happening inside the system. Both the web application and the API writes logs to standard output in their containers. These logs include HTTP requests as well as important actions like registration, login, posting cheeps, and following other users.
The logs are collected and brought together in one place. Docker handles them locally with rotation to avoid unlimited growth, and Alloy forwards them to Loki. Grafana is then used to visualize the logs. This makes it possible to investigate issues across the whole system instead of just a single machine. If something looks off in the metrics, the logs can be checked to find out why.

2.4 Security Hardening

Security is handled both in the pipeline and in the code. In the CI/CD process, static analysis tools, such as Trivy, help catch vulnerable code before being deployed. Container images are also scanned, and any issues are documented.
In production, traffic is protected using TLS at the load balancers, and HTTPS is enforced outside of development. Sensitive data like connection strings is provided through environment variables instead of being stored in the codebase. Data-protection keys are stored in the database so authentication remains stable across restarts and multiple instances. Containers also run without root privileges, which limits the impact if something goes wrong. These steps don't make the system fully secure, but they show that security has been taken into account throughout.

2.5 Availability and Scaling

Availability and scaling are handled through the infrastructure setup. Instead of running on a single server, the system uses two application instances behind two load balancers. Traffic is distributed using nginx and keepalived handles failover between load balancers. This means that if one component fails, another can take over.
Containers are configured with restart policies and dependencies to improve reliability. The application can also scale by adding more instances using the same setup, which makes it easier to handle increased load without changing the overall design. The main limitation is the database, which is still a more centralized component. Even so, the combination of multiple instances, load balancing, and failover provides availability and future scaling.

3 Reflection perspective

3.1 Evolution

3.1.1 Technical Debt

A major lesson from the project was how technical debt accumulates when minor deficiencies are deferred instead of being addressed early. As we based our MiniTwit on our earlier Chirp project from the BDSA course, this meant that we often trusted the existing components and did not revisit all parts of the codebase before implementing new features covered in the lectures, instead we prioritized progress over cleanup.
This became visible in several places. For example, when changing the database from SQLite to PostgreSQL, we left initialization code in the production code long after the system had moved to the PostgreSQL database. Also during the integration with the simulator, we never fixed the GET /latest endpoint, which returned 404 instead of a valid response if no simulator traffic had yet arrived. Individually, these issues seemed minor and harmless, and were therefore deprioritized in favor of more urgent tasks. The lesson learned was that technical debt rarely disappears on its own, once an issue is deprioritized, it often stays and continues to affect the system.

3.1.2 Refactoring

Rather than rewriting from scratch, the group chose to port the Chirp! codebase from the BDSA course (df2513d). This saved time initially, but the codebase carried assumptions that did not fit the DevOps course. It was originally built for SQLite and hosted on Azure as a single instance, so we switched it to PostgreSQL and containerized deployment on DigitalOcean. This meant that we had to refactor some of our database logic, since SQLite and PostgreSQL handle case sensitivity differently. We patched it for PostgreSQL, but this broke the unit tests since they were still running against SQLite. The fix was reverted (59f3f78, ec44939), and the underlying mismatch was never resolved. Similarly, wiring the application up to the simulator required a custom authentication handler (9c7dd39) and disabling all of ASP.NET Core Identity's password requirements (41a2e80), since the simulator registers users with very weak passwords.

3.2 Operation and Maintenance

3.2.1 Logging

Implementing logging challenged us initially. We had uncertainties about what to log and how to structure it. We updated our logging throughout, in order to collect all the necessary information. Looking back, we should have spent more time identifying what information would have been important to log, that could help us evaluate and improve the system later.

3.2.2 Grafana

We added monitoring to our application through Prometheus with Grafana for visualization. Up until the final weeks of the project, we had only implemented a global counter for HTTP requests, as well as a live status for the deployed website. Due to our limited number of dashboards, we didn't utilize monitoring to check the status of the website as much as we could have throughout the project.
We ended the project with the monitoring dashboards in (Figure 8 & Figure 9). Ideally, we would have expanded and implemented more consequential monitoring, such as CPU usage, but due to time constraints during the semester and our lack of experience with monitoring, we didn't make as extensive use of it as we could have.

3.3 DevOps Reflections

Working in a DevOps style changed how we approached the project compared to previous projects such as Chirp! from BDSA. One of the biggest differences was the deployment strategy. Instead of deploying by copying the entire codebase, we worked with Docker images, which was a valuable learning experience and made the process more consistent and reliable. However, we could have made better use of the tools we set up. For example, Grafana dashboards were created but not regularly checked and alerts were never set up.
Since we used Chirp!, the system was already relatively well-developed, and most of our work was thereby more operational rather than developmental, with the API being one of the few parts that was developed further. In the beginning much of the work had to be done collaboratively, since the tasks were tightly coupled, such as setting up CI/CD pipelines, introducing containerization with Docker and Vagrant, and getting the existing system into a deployable state. In practice much of our knowledge sharing happened through informal discussions rather than written documentation, which worked while the project was active, but made maintenance harder afterwards. The biggest lesson during the project has been that tools such as CI/CD, Docker, and monitoring are valuable, but only if they become part of the group's actual working habits.

4 Generative AI

For this project we have used a range of different AI tools, such as Github Copilot, ChatGPT, Claude and SonarQube AI analysis tools. During this course we mainly followed the extensive guides and examples from the lectures and exercises to complete tasks. AI was used to supplement that process when errors would occur or when we were missing information.

- **Writing code** - In this course we had to write a lot of commands and scripts in our Docker files, Vagrant files and other file types that we had never seen before. There were also instances where the guides were outdated or the examples from class weren't sufficient. In these instances the AI tools were used as a means of not wasting time looking for the correct information.

- **Code Analysis** - We used SonarQube as a static analysis tool. This is primarily not an AI tool, but it does use AI to find and offer solutions to the problems it detects during it's analysis.
- **Debugging** - Sometimes we would use the aforementioned AI tools to help find the cause of an error we got in the terminal.

The tools supported us in saving time when looking up the information we needed to complete a task. This made the process quicker and provided us with knowledge, which helped us improve our understanding of the code and the errors that would occur.

As described we used AI tools to gather information, but that only supports us if that information is correct. There were of course many suggestions made by the AI tools that did not work, and could therefore be seen as a waste of time. This is the only way we were hindered by the use of AI tools during this course.